

Insper

Engenharia de Computação
Computação em Nuvem

Relatório de Arquitetura

Plataforma Escalável em Nuvem para Processamento de Pagamentos

Primeiro Semestre de 2026

1. Introdução

Este relatório apresenta as decisões arquiteturais tomadas no desenvolvimento da Plataforma Escalável em Nuvem para Processamento de Pagamentos. O objetivo é documentar as escolhas técnicas realizadas, justificando cada componente da solução com base em critérios de escalabilidade, segurança, aderência a práticas do mercado e viabilidade técnica para um MVP.

O foco principal da arquitetura foi o uso de conceitos amplamente adotados no mercado de tecnologia, especialmente no setor de fintechs e sistemas financeiros distribuídos. Cada serviço da AWS escolhido reflete uma prática consolidada na indústria, garantindo que a experiência adquirida durante o projeto seja diretamente aplicável em contextos profissionais reais.

Além do aspecto funcional, a segurança da infraestrutura foi tratada como prioridade desde o início do projeto. A arquitetura foi desenhada para manter cada componente no ambiente mais restrito possível, minimizando a superfície de exposição e aplicando o princípio do menor privilégio em todas as camadas.

2. Visão Geral da Arquitetura

A arquitetura adota um modelo híbrido que combina dois paradigmas de computação em nuvem: containers gerenciados para o backend principal e computação serverless para o processamento de transações financeiras. Essa separação foi uma decisão deliberada, fundamentada no conceito de isolamento entre o plano de controle e o plano de transações, um padrão amplamente utilizado em plataformas financeiras modernas.

O sistema está organizado em dois caminhos distintos de comunicação: o caminho de operações comuns (CRUD de usuários, consultas e listagens), que trafega pelo Application Load Balancer até o NestJS rodando em container no ECS Fargate; e o caminho de transações financeiras (criação e processamento de pagamentos), que trafega pelo API Gateway até duas funções AWS Lambda dedicadas, com processamento assíncrono via fila SQS.

Essa separação garante que picos de carga em um caminho não afetem o outro. Se o fluxo de transações estiver sob estresse durante um teste de carga com JMeter, as operações de consulta e cadastro continuam funcionando normalmente, e vice-versa. Cada caminho escala de forma independente.

3. Camada de Frontend

3.1. React no Amazon S3 + CloudFront

O painel administrativo foi desenvolvido em React e hospedado como aplicação estática no Amazon S3, com o Amazon CloudFront como CDN na frente. O S3 armazena os arquivos gerados pelo build do React (HTML, CSS, JavaScript), e o CloudFront os distribui com HTTPS e cache em edge locations globais.

Justificativa: essa abordagem é o padrão da indústria para hospedar Single Page Applications na AWS. Não há servidor para gerenciar, o custo é quase zero para um MVP, e a escalabilidade é ilimitada, já que são apenas arquivos estáticos servidos por CDN. A escolha

do CloudFront sobre o hosting nativo do S3 se deu pela necessidade de HTTPS e pela melhoria de latência proporcionada pelo cache distribuído.

Alternativas descartadas: hospedar o frontend em uma instância EC2 com Nginx foi descartado por contradizer a filosofia de infraestrutura gerenciada adotada e por exigir gerenciamento de servidor. O AWS Amplify Hosting foi considerado, mas oferece menos controle sobre a configuração do CloudFront e das políticas de cache.

4. Camada de Backend Principal

4.1. NestJS no Amazon ECS Fargate

O backend principal da aplicação foi desenvolvido em NestJS (Node.js) e implantado como container Docker no Amazon ECS com modo de execução Fargate. O Fargate é um mecanismo serverless para containers: a aplicação é empacotada em uma imagem Docker, armazenada no Amazon ECR, e o Fargate gerencia toda a infraestrutura subjacente, incluindo provisionamento de máquinas, monitoramento e reinicialização automática de containers.

Justificativa: a escolha de manter o NestJS no ECS Fargate, separado das funções Lambda, foi motivada primariamente pela separação de responsabilidades. O ECS é responsável por tudo que é exibido constantemente no frontend: listagem de usuários, consulta de pagamentos, visualização de transações e acompanhamento de status. Essas operações são requisitadas o tempo todo pelo painel administrativo e, portanto, faz sentido que o servidor esteja sempre ativo, pronto para responder com baixa latência. Já as transações financeiras têm uma natureza diferente: elas acontecem por momento de trigger, quando o usuário decide criar um pagamento. Não é algo que ocorre a todo instante, e não faz sentido manter um servidor processando essa lógica continuamente. O Lambda é ideal para esse cenário porque ele acorda sob demanda, executa a transação e volta a ficar inativo.

Essa separação também garante independência entre os componentes: as funções Lambda existem e operam independentemente da instância do ECS estar ativa ou não. Se o container do NestJS for reiniciado, atualizado ou sofrer uma falha, o fluxo de transações via Lambda continua funcionando normalmente, pois são infraestruturas completamente desacopladas. O inverso também é verdadeiro: problemas nas Lambdas não afetam a disponibilidade do painel administrativo.

Do ponto de vista técnico, embora seja possível executar o NestJS em Lambda (via adaptadores como o serverless-express), o framework possui um processo de bootstrap pesado, com inicialização do container de injeção de dependências, carregamento de módulos e configuração de rotas. No Fargate, esse bootstrap acontece uma única vez, e o servidor permanece ativo reutilizando conexões e recursos, incluindo o pool de conexões persistente com o PostgreSQL. O Fargate foi escolhido em detrimento de ECS com EC2 por eliminar a necessidade de gerenciar instâncias, e a containerização via Docker é o padrão do mercado para deploy de aplicações backend.

4.2. Application Load Balancer (ALB)

O Application Load Balancer atua como ponto de entrada público para o backend NestJS. Ele fornece uma URL estável e fixa que o frontend utiliza para todas as chamadas HTTP de operações comuns. O ALB roteia as requisições para os containers Fargate, distribui carga

entre múltiplas instâncias quando o auto-scaling está ativo, e realiza health checks para garantir que o tráfego só seja direcionado a containers saudáveis.

Justificativa: o ALB é o ponto de entrada padrão para arquiteturas baseadas em containers na AWS. Os containers Fargate não possuem IP público fixo e podem ser substituídos a qualquer momento, tornando o ALB essencial como camada de abstração estável. A escolha do ALB sobre o API Gateway para este caminho se deu por custo (o ALB é significativamente mais barato para tráfego contínuo) e por integração nativa com o ECS.

5. Camada de Transações Financeiras

5.1. Amazon API Gateway

O caminho de transações financeiras utiliza o Amazon API Gateway como ponto de entrada dedicado. O API Gateway recebe as requisições de criação de pagamento e invoca a primeira função Lambda (Lambda de recepção). Ele oferece throttling nativo, o que permite limitar a taxa de requisições para proteger o sistema contra abuso.

Justificativa: a separação do caminho de transações em um API Gateway dedicado, distinto do ALB, reflete o padrão de isolamento entre plano de controle e plano de dados utilizado em plataformas financeiras. O API Gateway é o serviço nativo da AWS para integração com Lambda, e sua adoção neste caminho garante escalabilidade automática e independente do backend principal.

5.2. AWS Lambda (Duas Funções Distintas)

O processamento de transações é realizado por duas funções AWS Lambda separadas, cada uma com responsabilidade única e trigger próprio. Ambas as funções foram implementadas em Python. A primeira função Lambda é invocada pelo API Gateway: quando uma requisição de criação de pagamento chega, essa Lambda valida os dados, registra o pagamento com status PENDENTE no PostgreSQL e envia uma mensagem para a fila SQS, retornando imediatamente um HTTP 202 Accepted ao usuário. A segunda função Lambda é invocada pelo Amazon SQS: quando uma mensagem chega na fila, essa Lambda consome a mensagem, processa a transação financeira e atualiza o status no banco para APROVADO ou REJEITADO.

Escolha da linguagem: a decisão de utilizar Python nas funções Lambda, em vez de Node.js (que é a linguagem do backend NestJS), foi motivada pelo maior domínio da equipe sobre a linguagem e pela sua menor rigidez sintática, o que permitiu maior agilidade no desenvolvimento das funções serverless. O Python é uma das linguagens mais utilizadas em funções Lambda na AWS, com suporte nativo ao runtime e ampla disponibilidade de bibliotecas para integração com serviços como SQS, RDS e Secrets Manager. A utilização de linguagens diferentes no backend principal (Node.js) e nas Lambdas (Python) é uma prática comum em arquiteturas de microserviços, onde cada componente pode adotar a linguagem mais adequada ao seu contexto.

Justificativa: a separação em duas funções distintas foi uma decisão deliberada motivada por isolamento de responsabilidades. Cada função possui seu próprio código, sua própria configuração de memória e timeout, seus próprios logs no CloudWatch e sua própria role IAM com permissões mínimas. A Lambda de recepção pode ser configurada com timeout curto e pouca memória (pois só valida e enfileira), enquanto a Lambda de processamento pode ter

timeout mais longo e mais memória (pois executa a lógica da transação). Além disso, se a Lambda de processamento apresentar um bug, a Lambda de recepção continua funcionando normalmente, garantindo que nenhum pagamento se perca. Essa separação reflete o padrão de microserviços adotado em plataformas financeiras, onde cada componente é independente em deploy, monitoramento e escala.

5.3. Amazon SQS (Processamento Assíncrono)

O Amazon SQS atua como fila de mensagens entre a etapa de recepção e a etapa de processamento da transação. Esse desacoplamento é o núcleo do processamento assíncrono exigido pelo projeto. A fila foi configurada com uma Dead Letter Queue (DLQ), que captura mensagens que falharam após múltiplas tentativas de processamento, evitando perda de dados e permitindo análise posterior de falhas.

Justificativa: a fila garante três propriedades críticas para um sistema de pagamentos. Primeira, resiliência: se o processamento falhar, a mensagem retorna à fila e é processada novamente, sem perda de transações. Segunda, absorção de pico: durante testes de carga com JMeter, milhares de requisições simultâneas são absorvidas pela fila, permitindo que o processamento ocorra na velocidade que o sistema suporta. Terceira, desacoplamento: a API responde imediatamente ao usuário enquanto o processamento real acontece em background.

6. Camada de Persistência

6.1. Amazon RDS PostgreSQL

O banco de dados da aplicação é um PostgreSQL gerenciado pelo Amazon RDS. Todos os dados transacionais (usuários, pagamentos, histórico de transações) são armazenados nesta instância. O RDS cuida de backups automáticos, patches de segurança e recuperação em caso de falha.

Justificativa: a escolha de PostgreSQL (relacional) em vez de DynamoDB (NoSQL) foi motivada pela natureza do domínio de pagamentos, que exige consistência transacional forte (ACID), relações entre entidades (usuários, pagamentos, transações) e capacidade de queries relacionais complexas. O PostgreSQL é o banco de dados mais utilizado em fintechs e sistemas financeiros por oferecer essas garantias de forma nativa.

6.2. Amazon RDS Proxy

O RDS Proxy foi implementado como intermediário entre as funções Lambda e o PostgreSQL. Ele realiza connection pooling, recebendo centenas de conexões simultâneas das Lambdas e gerenciando um pool reduzido de conexões reais com o banco de dados.

Justificativa: cada invocação simultânea do Lambda abre uma conexão independente com o PostgreSQL. O PostgreSQL possui um limite fixo de conexões simultâneas (tipicamente 80 a 200, dependendo da instância). Durante os testes de carga com JMeter, centenas de invocações simultâneas do Lambda estourariam esse limite, causando erros de "too many connections" e indisponibilidade do banco. O RDS Proxy resolve esse problema afunilando as conexões: 300 conexões do Lambda são traduzidas em apenas 20 a 30 conexões reais com o banco. O NestJS, por ser um processo contínuo que mantém seu próprio pool interno de conexões, acessa o PostgreSQL diretamente sem necessidade do Proxy.

7. Segurança e Rede

7.1. VPC Customizada

Toda a infraestrutura foi implantada em uma VPC (Virtual Private Cloud) customizada, criada especificamente para o projeto. A VPC foi configurada com subnets públicas (para o ALB, que precisa receber tráfego da internet) e subnets privadas (para o NestJS no Fargate, as funções Lambda, o RDS PostgreSQL e o RDS Proxy).

Justificativa: a criação de uma VPC customizada em vez do uso da VPC padrão da AWS foi uma decisão motivada por segurança. A VPC padrão possui apenas subnets públicas, o que exporia o banco de dados e os containers à internet. Com a VPC customizada, o PostgreSQL e o NestJS ficam em subnets privadas, inacessíveis diretamente da internet, e o único ponto de exposição pública é o ALB. Essa separação entre subnets públicas e privadas é uma prática de segurança padrão do mercado.

7.2. VPC Endpoints

Para permitir que os recursos nas subnets privadas se comuniquem com serviços da AWS (como SQS e CloudWatch), foram utilizados VPC Endpoints em vez de um NAT Gateway. Os VPC Endpoints criam uma conexão privada entre a VPC e o serviço da AWS, sem que o tráfego precise sair para a internet pública. A comunicação entre as Lambdas e o RDS, bem como entre o ECS e o RDS, também foi configurada via VPC Endpoints, mantendo todo o tráfego dentro da rede interna da AWS.

Justificativa: a alternativa ao VPC Endpoint seria o NAT Gateway, que roteia o tráfego das subnets privadas através da internet pública para alcançar os serviços da AWS. O NAT Gateway tem um custo fixo de aproximadamente US\$ 32 por mês por Availability Zone, além de cobrança por GB de dados processados. Para um projeto acadêmico com orçamento limitado, esse custo é significativo. Os VPC Endpoints têm um custo inferior e, além da economia, oferecem uma vantagem de segurança: o tráfego nunca sai da rede da AWS, eliminando a exposição à internet pública. Essa escolha demonstra capacidade de avaliar trade-offs entre custo e arquitetura de rede.

7.3. Security Groups

Cada componente da arquitetura possui seu próprio security group, configurado com o princípio do menor privilégio. O security group do PostgreSQL permite conexões na porta 5432 apenas dos security groups do NestJS (Fargate) e do RDS Proxy, bloqueando qualquer outro acesso. O security group do ALB permite tráfego HTTP/HTTPS da internet. O security group do Fargate permite tráfego apenas vindo do ALB.

7.4. Acesso ao Banco de Dados para Desenvolvimento

Durante o desenvolvimento, foi necessário acessar o banco de dados PostgreSQL a partir das máquinas locais da equipe para validação de dados, debugging e execução de migrações. Para isso, foi adicionada uma inbound rule no security group do RDS, liberando a porta 5432 exclusivamente para o IP público da máquina de desenvolvimento. Essa regra permite que ferramentas como DBeaver ou pgAdmin se conectem ao banco, mantendo a restrição de acesso apenas ao IP específico do desenvolvedor.

Observação de segurança: essa regra de acesso por IP é uma prática aceitável para ambientes de desenvolvimento e MVP, mas não seria recomendada em produção. Em um cenário produtivo, o acesso ao banco seria feito exclusivamente através de bastion hosts ou AWS Systems Manager Session Manager, sem exposição direta da porta do banco, mesmo com restrição de IP. Para o escopo deste projeto, a abordagem adotada oferece o equilíbrio adequado entre segurança e produtividade da equipe.

8. Observabilidade

A camada de observabilidade é composta pelo Amazon CloudWatch, que coleta automaticamente os logs de todas as funções Lambda e containers ECS, além de métricas como latência, taxa de erro, número de invocações e profundidade da fila SQS. Dashboards personalizados foram criados para consolidar as métricas mais relevantes em uma visão unificada, facilitando a análise do comportamento do sistema durante os testes de carga.

Justificativa: a observabilidade é essencial para a análise dos testes de carga com JMeter. As métricas do CloudWatch fornecem evidências quantitativas do comportamento do sistema sob estresse, permitindo identificar gargalos, medir latência por componente e validar a eficácia do processamento assíncrono. Como evolução futura, a ativação do AWS X-Ray permitiria rastrear requisições individuais através de todos os componentes da arquitetura, gerando mapas visuais de latência por serviço.

9. Dificuldades Enfrentadas

O desenvolvimento do projeto envolveu desafios técnicos significativos, muitos deles decorrentes do contato inicial com serviços da AWS que não eram previamente conhecidos pela equipe. Esta seção documenta as principais dificuldades enfrentadas e como foram superadas.

9.1. Amazon ECR e o Fluxo de Deploy no ECS

Uma das primeiras dificuldades encontradas foi o desconhecimento do Amazon ECR (Elastic Container Registry), o serviço da AWS responsável por armazenar imagens Docker. Para que o ECS Fargate consiga rodar um container, a imagem Docker precisa estar disponível em um registry acessível, e o ECR é o registry nativo da AWS para esse fim. Sem entender essa peça, o fluxo completo de deploy do NestJS no ECS não fazia sentido: era necessário primeiro construir a imagem Docker localmente, autenticá-la no ECR, fazer o push da imagem, e só então configurar o ECS para utilizá-la. A compreensão desse fluxo (build → push para ECR → ECS puxa a imagem → Fargate executa o container) foi fundamental para destravar o deploy do backend.

9.2. Target Groups e Services no ECS

Outro ponto de dificuldade foi compreender os conceitos de Target Group e Service no contexto do ECS com ALB. O Target Group é um recurso do ALB que define para onde as requisições devem ser encaminhadas: ele agrupa os containers que estão aptos a receber tráfego e mantém o registro de quais estão saudáveis através de health checks periódicos. Já o Service no ECS é o recurso que garante que um número desejado de containers esteja sempre rodando: ele monitora o estado dos containers, reinicia os que falharem e registra novos containers no Target Group do ALB automaticamente. A conexão entre esses dois conceitos

não era óbvia inicialmente, e entender que o Service é quem mantém os containers vivos enquanto o Target Group é quem diz ao ALB para onde mandar o tráfego foi essencial para configurar a infraestrutura corretamente.

9.3. Migrações do Backend para o Banco de Dados

A execução das migrações do banco de dados foi uma fonte recorrente de problemas durante o desenvolvimento. Erros e inconsistências no código do backend geravam falhas nas migrações, que por sua vez deixavam o schema do banco em estados parciais difíceis de corrigir. A depuração exigia entender simultaneamente o estado do banco, o estado das migrações pendentes e o código que as gerou.

Essa dificuldade reforçou a importância conceitual do Dockerfile como ferramenta de reprodutibilidade. O Dockerfile define o ambiente de execução da aplicação de forma determinística: sistema operacional, versão do Node.js, dependências instaladas e comando de inicialização. Com um Dockerfile bem configurado, o ambiente é sempre o mesmo, independente de onde o container roda, eliminando problemas do tipo "funciona na minha máquina mas não funciona na AWS". Essa compreensão de que o Docker resolve o problema de consistência de ambiente foi um aprendizado relevante do projeto.

9.4. VPC Endpoints: Descoberta e Decisão de Custo

Uma dificuldade que surgiu durante a configuração da rede foi descobrir que os recursos nas subnets privadas (Lambda e ECS) não conseguiam se comunicar com o RDS e com serviços da AWS sem uma rota de saída configurada. A solução mais comum para isso é o NAT Gateway, que permite que recursos em subnets privadas acessem a internet. Porém, ao analisar os custos, identificou-se que o NAT Gateway cobra aproximadamente US\$ 32 por mês por Availability Zone, além de US\$ 0,045 por GB de dados processados. Para um projeto acadêmico, esse custo era proibitivo.

A alternativa encontrada foi o uso de VPC Endpoints, que criam uma conexão privada direta entre a VPC e o serviço de destino, sem passar pela internet. O tráfego permanece inteiramente dentro da rede da AWS, o que além de ser mais barato, é mais seguro. A descoberta dessa solução exigiu pesquisa sobre as opções de conectividade privada da AWS e compreensão das diferenças entre Interface Endpoints e Gateway Endpoints. Essa decisão representou um aprendizado importante sobre análise de custo em arquiteturas de nuvem.

9.5. Integração com Supabase para Autenticação

Durante o planejamento do projeto, foi identificada a necessidade de um sistema de autenticação para controlar o acesso ao painel administrativo e proteger os endpoints da API. A solução inicialmente planejada era a integração com o Supabase Auth, um serviço de autenticação gerenciado que oferece funcionalidades prontas como cadastro de usuários, login, gestão de sessões e tokens JWT, eliminando a necessidade de implementar toda essa camada do zero.

Porém, essa integração não foi implementada por dois motivos. Primeiro, a falta de tempo frente às demais demandas do projeto, que já envolvia a configuração de múltiplos serviços AWS (ECS, Lambda, SQS, RDS, VPC, ALB, API Gateway). Segundo, o desconhecimento inicial da equipe sobre como integrar um serviço externo como o Supabase dentro da arquitetura AWS montada, especialmente no que diz respeito à validação de tokens JWT nas duas camadas de entrada (ALB para o NestJS e API Gateway para as Lambdas).

É importante registrar que em um ambiente de produção, a implementação de uma camada de autenticação seria essencial. Sem ela, qualquer pessoa com acesso à URL da API poderia criar pagamentos, consultar transações e manipular dados. O Supabase Auth ou alternativas como o Amazon Cognito seriam soluções adequadas para esse requisito, e sua integração permanece como evolução prioritária da plataforma.

10. Resumo dos Componentes

Componente	Serviço AWS	Função na Arquitetura
Frontend	S3 + CloudFront	Hospedagem de arquivos estáticos do React com HTTPS e cache global
Backend API	ECS Fargate + ALB	NestJS em container para CRUD, consultas e operações comuns
Transações	API Gateway + 2 Lambdas	Recepção e processamento de pagamentos com funções independentes
Fila	Amazon SQS	Desacoplamento assíncrono entre recepção e processamento
Banco de dados	RDS PostgreSQL	Persistência relacional com garantias ACID
Connection pool	RDS Proxy	Gerenciamento de conexões entre Lambda e PostgreSQL
Rede	VPC customizada	Isolamento de rede com subnets públicas e privadas
Conectividade	VPC Endpoints	Comunicação privada entre serviços sem NAT Gateway
Observabilidade	CloudWatch	Logs, métricas, alarmes e evidências para testes de carga

11. Conclusão

A arquitetura apresentada neste relatório foi projetada com foco em dois pilares: aderência a práticas consolidadas do mercado e segurança da infraestrutura. Cada componente foi escolhido com base em justificativas técnicas claras, considerando trade-offs entre simplicidade, custo, escalabilidade e robustez.

A separação entre o plano de controle (NestJS no Fargate) e o plano de transações (Lambda + SQS) reflete um padrão arquitetural real utilizado em fintechs. O uso de VPC customizada com subnets privadas demonstra preocupação com segurança desde o desenho da solução. E a camada de observabilidade com CloudWatch fornece as evidências necessárias para validar o comportamento do sistema sob estresse.

O resultado é uma solução que não apenas atende aos requisitos funcionais do MVP, mas também demonstra a capacidade de projetar, implementar e defender tecnicamente uma arquitetura distribuída em nuvem com maturidade de engenharia.